EDUGENIE: GOOGLE GEMINI-POWERED LEARNING ASSISTANT

Project Name: edugenie google gemini powered learning assistant

Date: 07/07/26

Team Id: XXXXXX

Project Description:

EduGenie harnesses Generative AI to provide intelligent educational content generation, offering users a fast and personalized learning experience. The platform includes a Pathway Creator module, which produces three unique, custom learning roadmaps per request, a Smart Summary Generator, which delivers ten relevant key takeaways blending fundamental and deep theoretical concepts, and an Interactive Quiz Engine, which delivers three action-oriented assessment questions tailored to the selected educational level and subject.

Utilizing Google Gemini's API with the advanced Gemini 1.5 Pro model, EduGenie processes user inputs to deliver fast, high-quality learning materials. Built with Flask and deployed publicly via Ngrok, the platform ensures a seamless and user-friendly experience. With secure API key management via .env and responsible input handling, EduGenie empowers students, creators, and educational institutions to craft compelling learning paths with confidence.

Scenarios

Scenario 1: A user describes a topic on sustainable architecture and selects Undergraduate with a Casual tone. The system generates three descriptive learning paths optimized for visual and interactive audiences, ten relevant topic breakdowns mixing broad and niche concepts, and three punchy check-your-understanding questions all in seconds.

Scenario 2: A user promoting an advanced deep learning framework selects Professional with an Analytical tone. EduGenie generates rigorous, research-style curriculum matrices with a value-driven narrative, industry-relevant textbook references, and task lists designed to drive professional mastery such as mini-project sign-ups or math-heavy derivation reads.

Scenario 3: A teacher running a weekend cram session selects K-12 with an Engaging tone. The AI generates motivation-driven outlines filled with conceptual analogies, a curated terminology set targeting core exam metrics, and immediate assessment call-outs like "Solve this card before scrolling!" to maximize retention.

Technical Architecture Description:

EduGenie utilizes a modular three-tier architecture to deliver rapid, AI-driven educational content generation. The frontend provides a responsive user interface built with HTML and CSS while the Flask-based backend orchestrates data validation and prompt construction. It interfaces with the Google Gemini API using the Gemini 1.5 Pro model to generate tailored learning materials, with secure deployment managed locally via Flask and exposed publicly through Ngrok.

Pre-requisites:

- Flask Framework Knowledge: Flask Documentation
- Google Gemini API Familiarity: Gemini API Documentation
- HTML, CSS, and JavaScript Skills: W3Schools HTML/CSS/JavaScript Tutorials
- Python Programming Proficiency: Python Documentation
- Version Control with Git: Git Documentation
- Development Environment Setup: Flask Installation Guide
- Ngrok for Public Deployment: Ngrok Documentation

Project Workflow:

Milestone 1: Model Selection and Architecture

Activity 1.1: Generate a Google Gemini API Key

Before the application can communicate with the Gemini LLM, you need a valid Google Gemini API key. Follow the steps below to create one and connect it securely to the project.

> **Step 1 - Create a Google AI Studio Account:** Visit the official Google AI Studio interface and sign up for a free developer account using your Google account or institutional email address.

> **Step 2 - Navigate to API Keys:** After logging in, click on the prominent sidebar icon or menu section labeled "Get API Key" from the management panel layout.

> **Step 3 - Create a New API Key:** Click the "Create API Key" button. Give your key a descriptive name (e.g., edugenie-key) so it is easy to identify in your workspace later.

> **Step 4 - Copy the Key:** Click on "Submit" to acquire your unique key token. Copy it immediately and store it in a safe place, as you will not be able to view it again after closing the dialog.

> **Step 5 - Add the Key to Your Project:** Create a .env file in the root of your project directory (if it does not already exist) and paste the key as shown below:

```
GEMINI_API_KEY=your_copied_api_key_here
```

Step 6 - Verify the Key is Loaded: The app.py file loads this key automatically at startup using python-dotenv:

```
from dotenv import load_dotenv
import google.generativeai as genai

load_dotenv()
genai.configure(api_key=os.environ.get("GEMINI_API_KEY"))
```

Important - Never Commit Your API Key: Add .env to your .gitignore file to prevent accidentally pushing the secret configuration key to a public repository:

```
.env
```

Activity 1.2: Research and Select the Appropriate Generative AI Model

Here are the steps needed to select the best model for the project framework:

- Understand the Project Requirements: Review the specific needs of the EduGenie application, focusing on the types of structural content it will generate (pathways, summaries, and assessment quizzes).
- Explore Google Gemini's Model Documentation: Visit the Google developer documentation to examine the available Gemini models, including their context sizes, pricing tiers, operational tokens, and latency benchmarks.
- Evaluate Model Performance: Compare models based on pedagogical response structure, text-reasoning quality, and creative parsing suitability for structured training tasks.
- Select the Optimal Model: Choose gemini-1.5-pro for its deep multi-turn reasoning capabilities, set with a temperature of 0.7 and max_tokens parameters optimized for complex structural curriculum output.

Activity 1.3: Define the Architecture of the Application

- Draft an Architectural Diagram: Create a visual block depiction showing interactions between the frontend (HTML, CSS, JavaScript), the Flask server controller middleware, and the Google Gemini endpoint.
- Detail Frontend Functionality: Outline how users input target subjects, toggle learning levels (K-12, Undergraduate, Professional), and adjust the material tone parameters.
- Outline Backend Responsibilities: Specify how the Flask routing layer receives POST packets on / generate, validates content parameters, creates formatted prompt blocks, and safely passes requests forward.

- Describe AI Integration Points: Define how the backend constructs system configuration dict arrays like DIFFICULTY_MAPPINGS, manages model client callbacks, processes underlying JSON code objects, and returns structural elements.

Activity 1.4: Set Up the Development Environment

Isolate and install critical execution environment configurations using python package management targets:

```
D:\projects> python -m venv edugenie_env
D:\projects> "d:\projects\edugenie_env\Scripts ctivate"
(edugenie_env) D:\projects> pip install Flask==3.0.3 google-generativeai python-
dotenv==1.0.1
```

Milestone 2: Core Functionalities Development

Activity 2.1: Develop Core Content Generation Features

- Pathway Creator: Produces structural roadmap trees mapping linear execution blocks directly.
- Smart Summary Generator: Formulates a concise textual extraction matrix covering fundamental criteria headers.
- Interactive Quiz Engine: Generates structural questions complete with comprehensive contextual explanation payloads.

Milestone 3: App.py Development

Activity 3.1: Writing the Main Application Logic in app.py

Configure routing structures and prompt construction to stream valid data objects back safely to the client side interface:

```
@app.route("/generate", methods=["POST"])
def generate():
    data = request.get_json()
    topic = data.get("topic", "").strip()
    level = data.get("level", "undergraduate").lower()

    if not topic:
        return jsonify({"error": "Topic is required."}), 400

    prompt = build_educational_prompt(topic, level)
    model = genai.GenerativeModel('gemini-1.5-pro')
    response = model.generate_content(prompt)
    return jsonify({"success": True, "data": response.text})
```

Milestone 4: Frontend Development

Activity 4.1: Designing and Developing the User Interface

Establish a single-page dark glassmorphism layout, map key visual cards to specific topic outputs, embed copy-to-clipboard functionality, and integrate asynchronous fetch hooks for uninterrupted loading animations.

Milestone 5: Deployment

Activity 5.1: Preparing and Testing Local Flask Server Environment

```
(edugenie_env) D:\projects> python app.py
 * Running on http://127.0.0.1:5050
```

Activity 5.2: Public Deployment via Ngrok Secure Tunnels

```
D:\projects> ngrok http 5050
```

Conclusion:

EduGenie is a generative AI platform built with Flask and styled with a glassmorphism UI that streamlines customized educational curriculum development. It uses Google Gemini's high-speed reasoning interface to deliver structural learning pathways, concise summary extractions, and interactive concept diagnostics. This operational workflow validates how targeted AI integration can drastically reduce curriculum preparation thresholds, laying groundwork for future functional updates like multi-language scale metrics and permanent profile authentication structures.